CS6037 Machine Learning Assignment 4

Joe Ferguson, Max Eichelberger

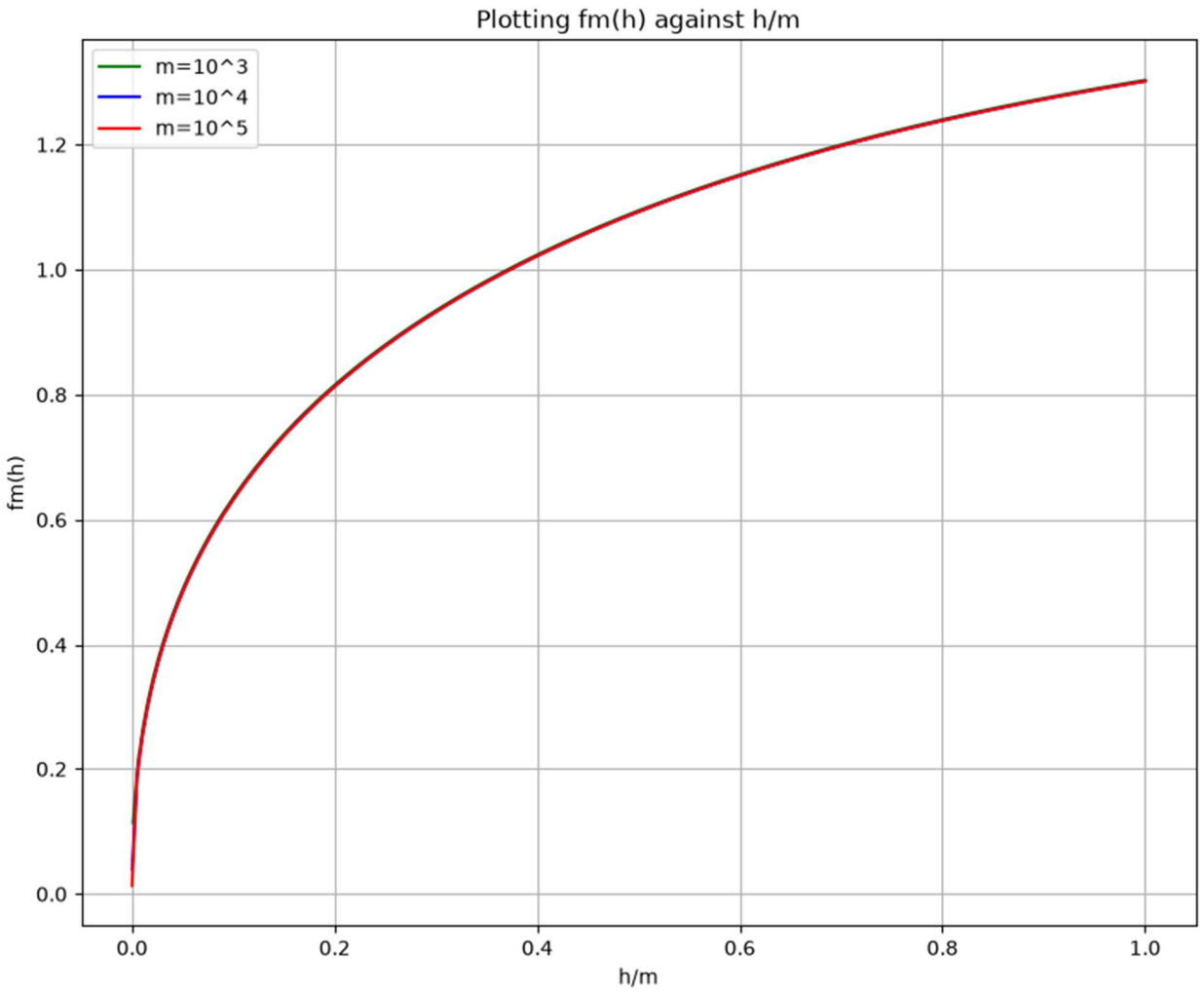
Contribution:

Joe wrote part 1 – programming and the bonus 1, Max wrote the essay.

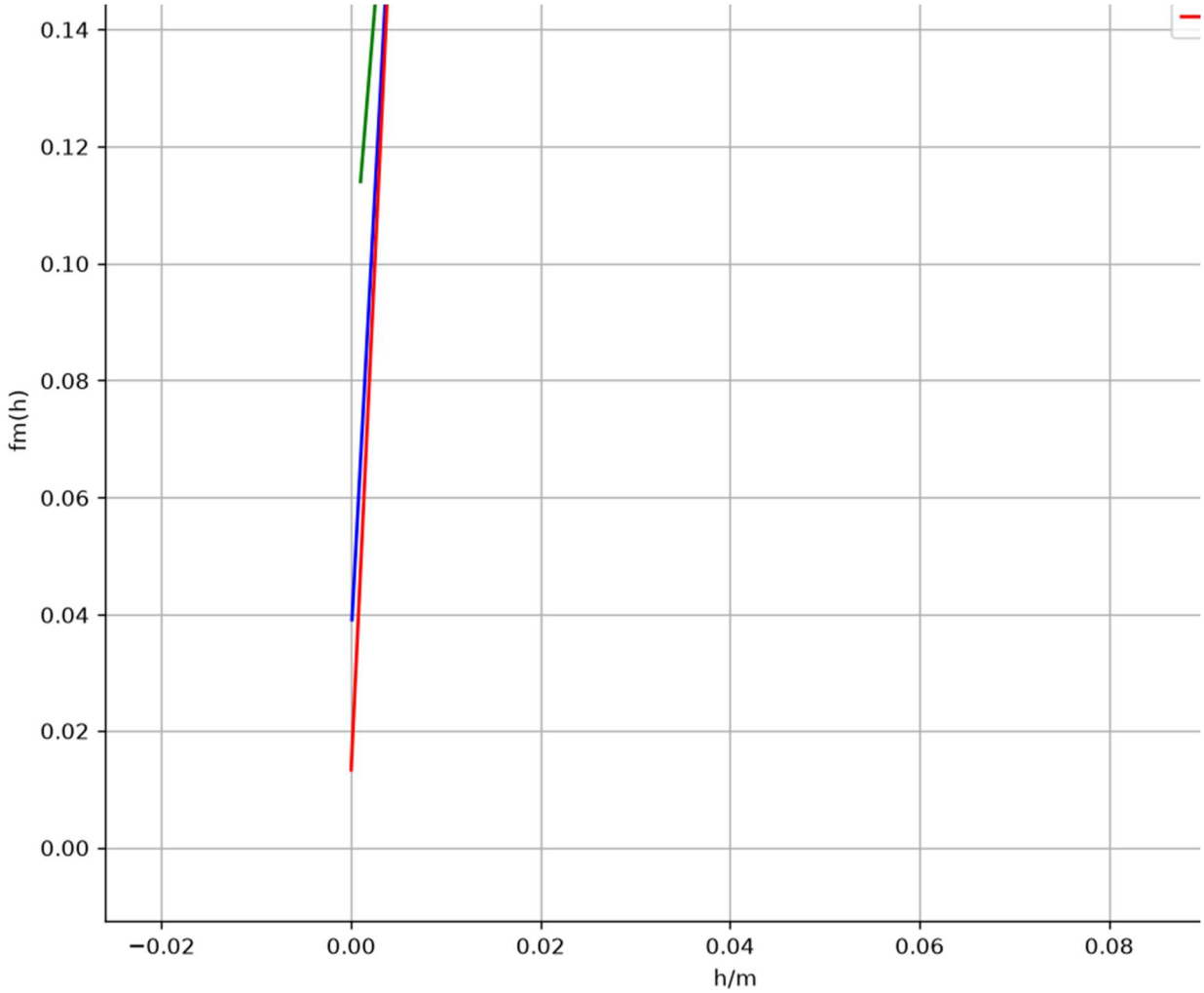
Part 1 – Programming - Joe

P1 – The VC Confidence Term

a)



All three lines are very similar, zooming in near (0,0) shows us:



So we can see they are indeed separate lines.

b)

```
h/m ratio for fm(h)=1 : m=1000, h/m=0.3708
h/m ratio for fm(h)=1 : m=10000, h/m=0.3731
h/m ratio for fm(h)=1 : m=100000, h/m=0.3733
```

As shown the results agree with the ratio h/m of approximately 0.37. To show why the ratio barely relies on m, we can write in terms of r=h/m:

$$f_m(h)^2 = \frac{h\left(\ln\frac{2m}{h} + 1\right) - \ln\frac{\eta}{4}}{m}$$

Which simplifies to:

$$f_m(h)^2 = \frac{h\left(\ln\frac{2m}{h}+1\right)}{m} - \frac{\ln\frac{\eta}{4}}{m}$$

Substituting r=h/m:

$$f_m(h)^2 = r\left(\ln\frac{2}{r}+1\right) - \frac{\ln\left(\frac{\eta}{4}\right)}{m}$$

Then we can see only the last term depends on m, which approaches 0 as m increases. It will then depend solely on the first term, which depends on the ratio picked.

c)

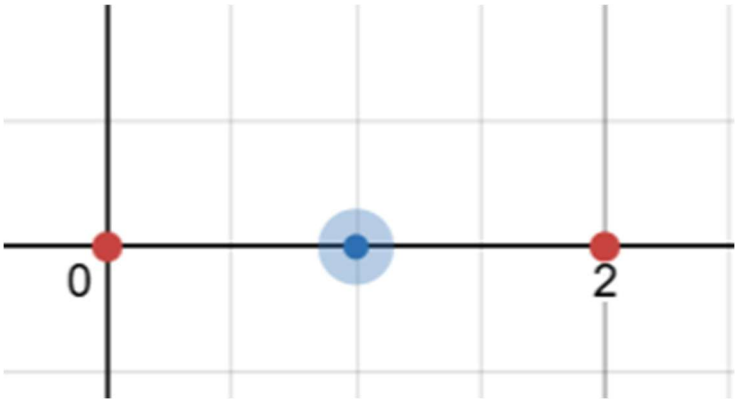
```
For Remp = 0, largest h = 11
For Remp = 0.05, largest h = 2
```

From these values, we can tell the VC bound is quite restrictive, for R_emp=0 it only allows a dimension at most 11 to guarantee the risk bound, for R_emp=0.05, it only allows for a dimension of at most 2. This is quite small to guarantee the risk.

P2 – A Shattering Checker

a)

```
Non-collinear: True
Collinear: False
```



From looking at the points, there is no line that exists that can separate the labels. The VC

dimension only guarantees that there exists at least one configuration that can be shattered, so even though this configuration can not be shattered the guarantee holds true.

b)

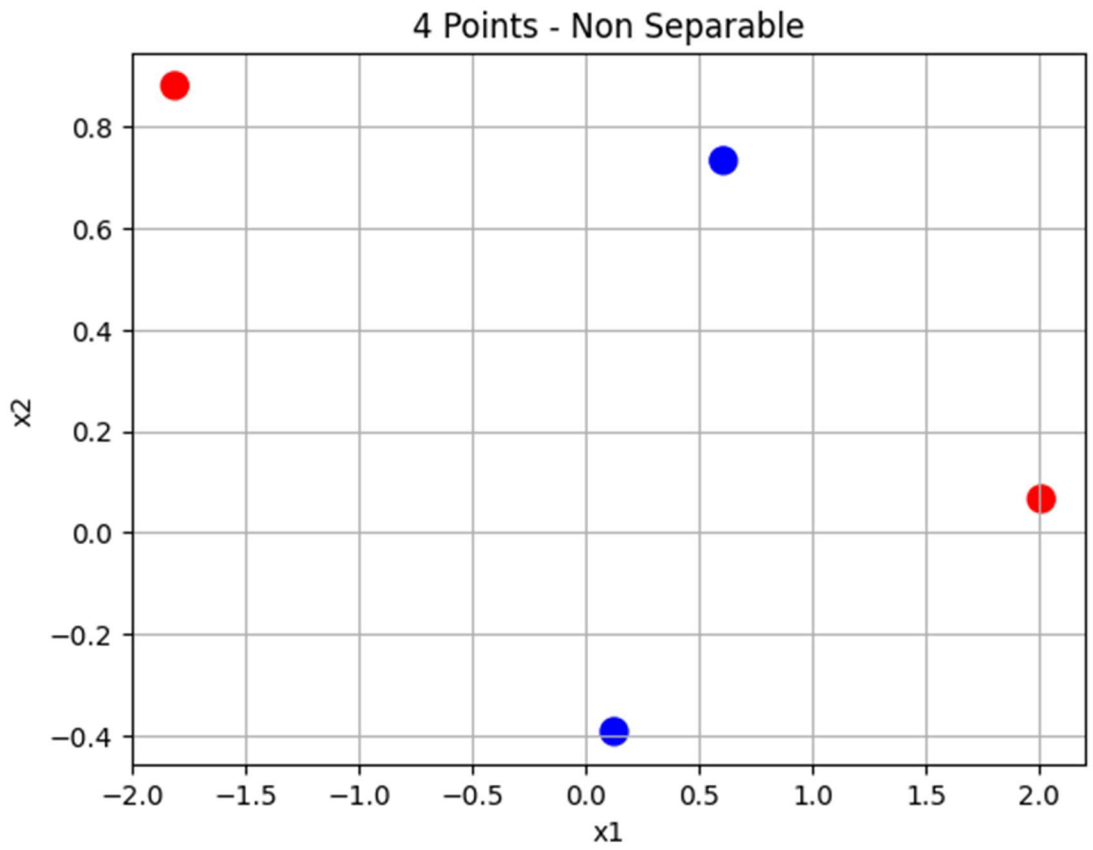
Shatterability in Dimension

N	N+1 Points	N+2 Points
2.0	1.0	0.0
3.0	1.0	0.0
4.0	1.0	0.0
5.0	1.0	0.0

As we can the results agree that the affine hyperplanes in R^n have VC dimension n+1, since the n+1 points shatter which is guaranteed. The n+2 points, which are not guaranteed, are not shattered.

c)

```
Random Points (4,2): [[ 0.60792399  0.73387609]
 [-1.81389528  0.88170483]
 [ 2.01108221  0.06662666]
 [ 0.12569227 -0.39224077]]
Non-separable Labelings: [array([-1,  1,  1, -1]), array([ 1, -1, -1,  1])]
```



As we can see it falls into the “If the four points form a convex quadrilateral, the “diagonal” labeling (opposite corners share a label) and its negation cannot be realized.” Case. Comparing the printed coords and labels with the plot allows us to validate this.

d)

```
✓ ### d - check 4 points - Joe ···

Points: [[-1  0]
 [ 1  0]
 [ 0 -1]
 [ 0  1]], are shattered: True


✓ ### d - check 1000 5-point sets - Joe ···

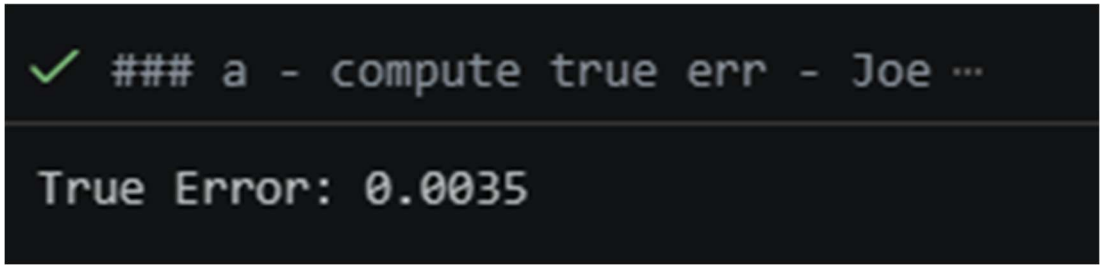
True: None of 1000 - (5,2) sets got shattered
```

P3 - PAC learning in action: rectangles and thresholds

a)

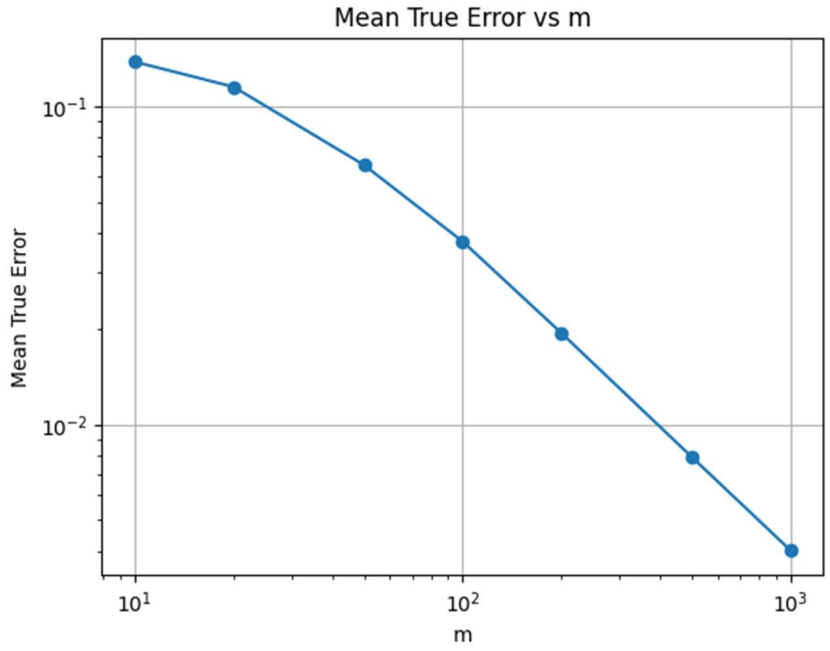
Since the learner constructs the tightest h_hat, it must lie within the target rectangle c, and all positive training points are correctly classified (same with negative training points), making it a consistent learner.

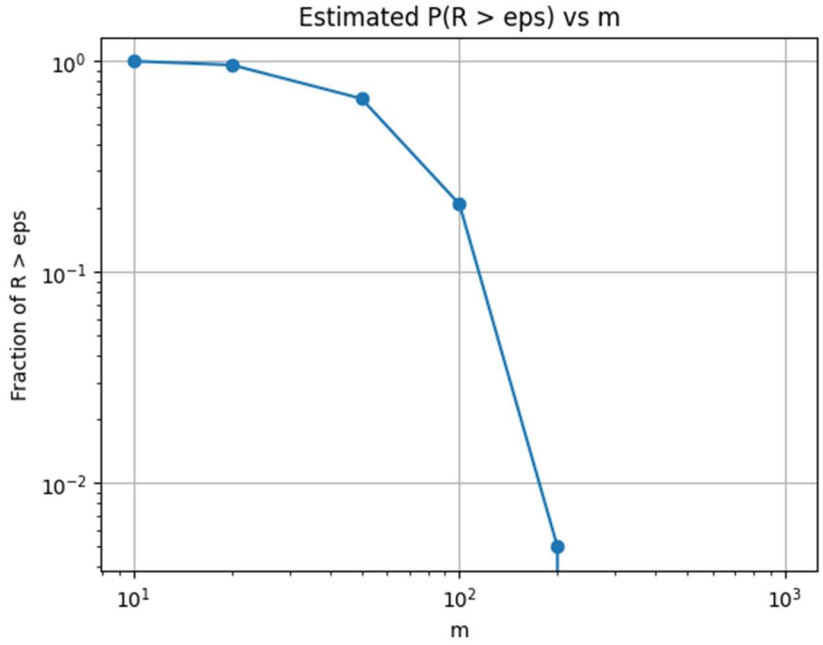
For the true error, the only error that can occur is the positive points outside the tight learned rectangle, and since the positive points extend no farther than the target rectangle c, the error would be area of the region inside the target c but outside h. We know the area of c = 0.15, thus the formula is, $R = \text{area}(c^*) - \text{area}(\hat{h}) = 0.15 - \text{area}(\hat{h})$, as shown in the problem description.



From this we could say, from 0.15 -x=0.0035, where x=0.1465 and is the area of the learned h_hat.

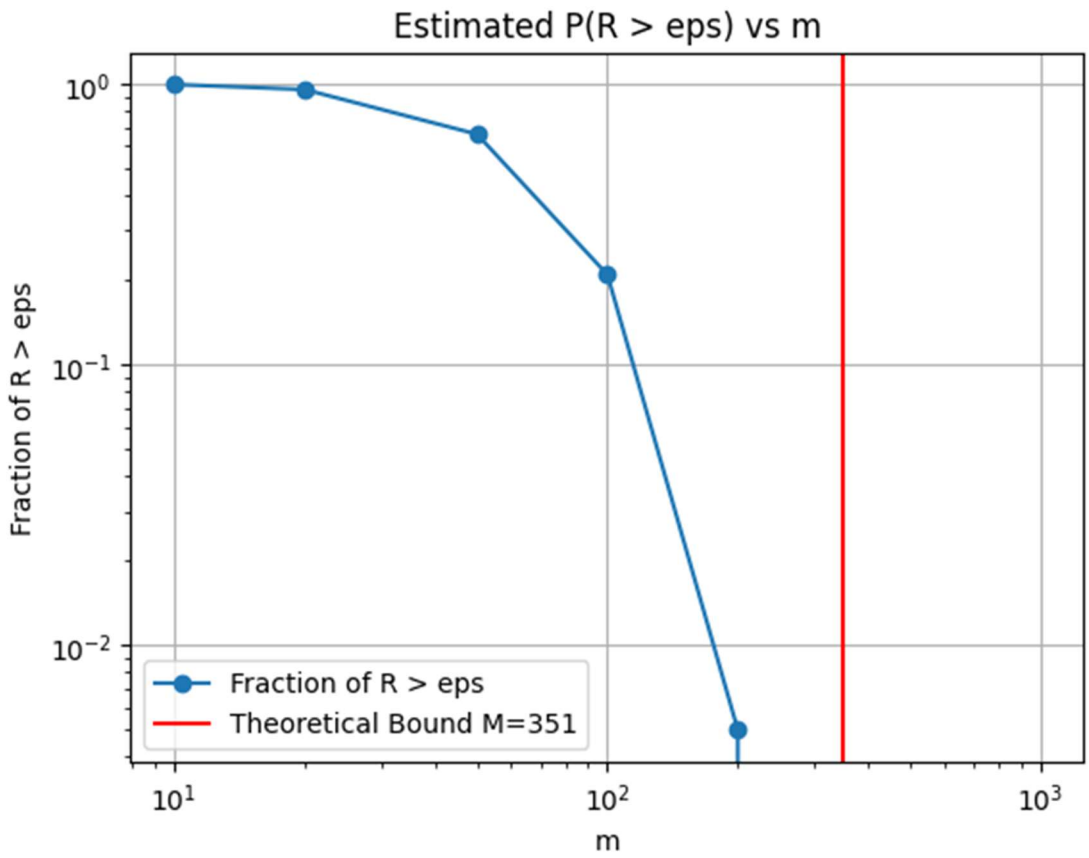
b)





c)

Plugging the values into a calculator, the result gives 350.56 which rounds to 351.




```
✓ ### c - find smallest m (to nearest 10) - Joe ···

Lowest M - to 10s : 150
Empirical Probability: 0.05000
Pessimistic Factor: 2.34000
```

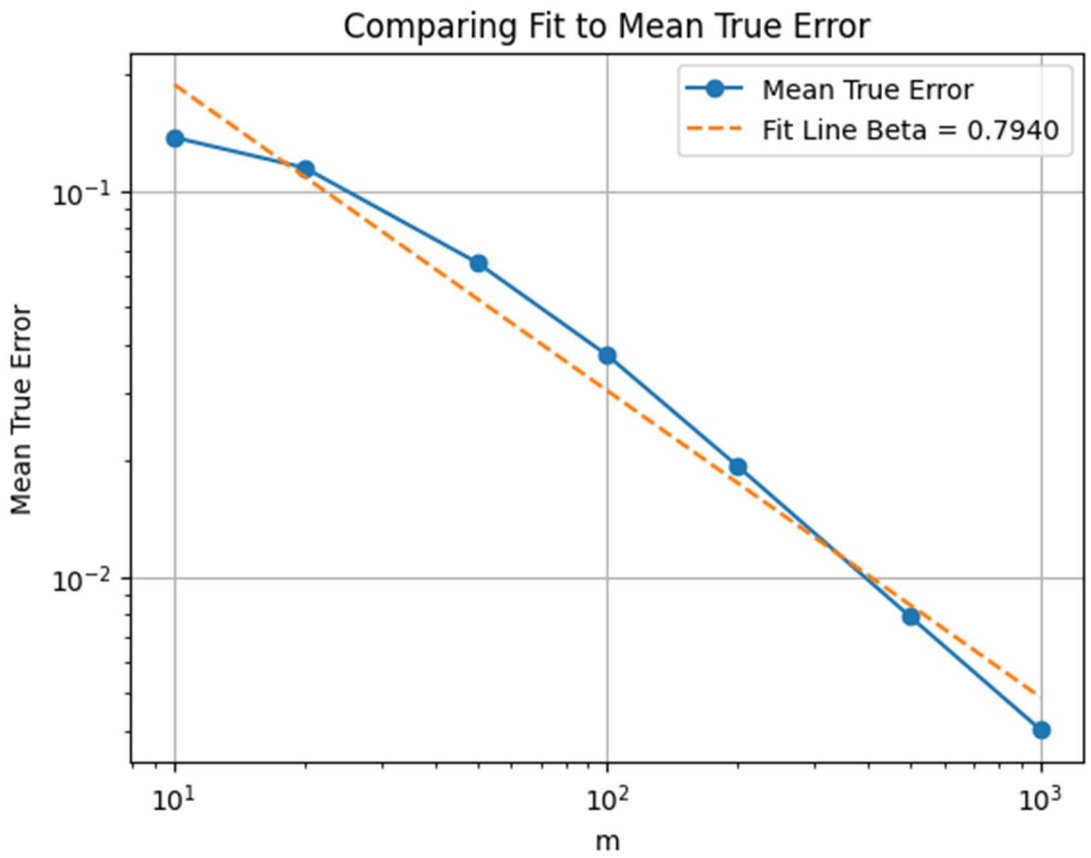
Shown above the computed M is 150, which means the theoretical bound is pessimistic by a factor of 2.34.

d)

```
✓ ### d - fit straight line of mean vs err = Joe ···

Beta : 0.7940
```

As shown above the rate is m^-0.794, which is closer to 1/m then 1/sqrt(m). From (2) the predicted one is 1/m, and from (1) the predicted one from (2) is 1/sqrt(m), which is easily determined from the square root in (1). These ignore the constant factors.



P4. Structural Risk Minimization versus cross-validation

a)

Using the hint that a nonzero polynomial of degree at most d has at most d real roots. A labeling following (+1,-1,+1,-1,...) needs d+1 sign changes, which a polynomial of degree d cannot achieve. Thus the VC dimension is at most d+1. This is the upper bound.

For d+1 points, we can use the polynomial roots by placing them in between where each sign changes. Since there are at most d sign changes we have enough (polynomial of degree d) to ensure this. This is the lower bound.

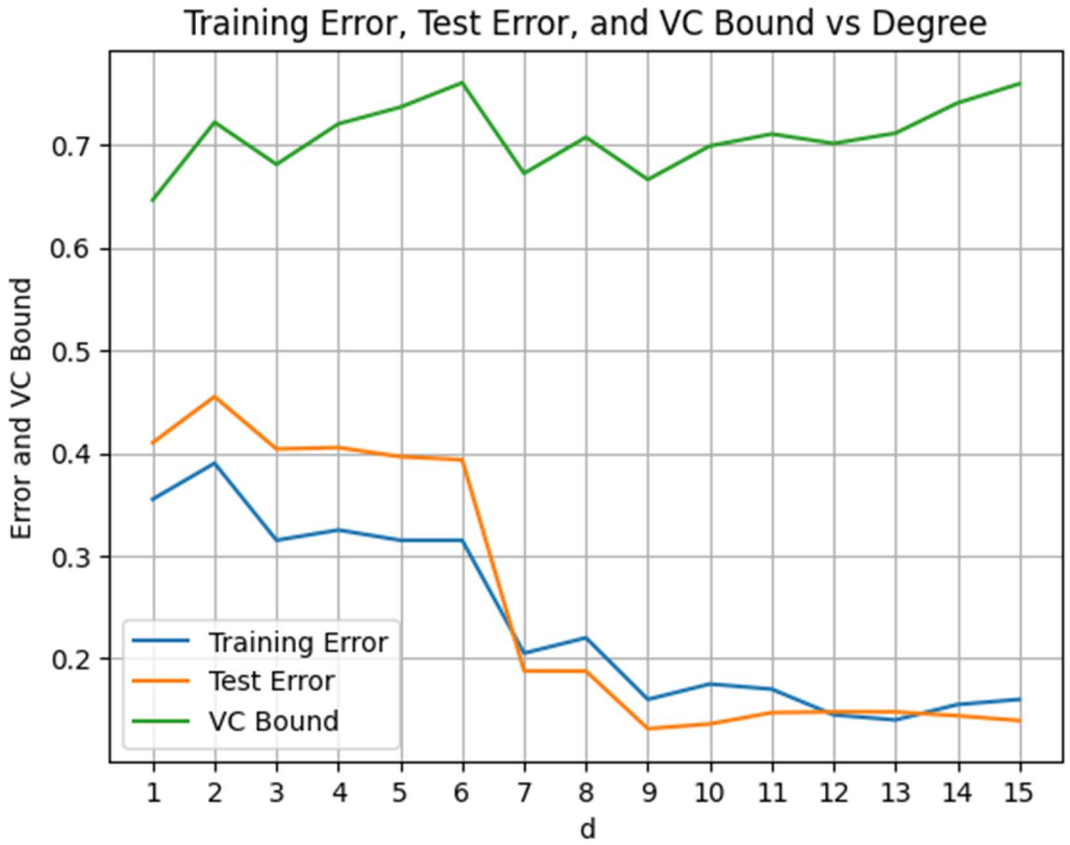
Both are combined to ensure the VC dimension of degree d polynomial classifiers is equal to d+1.

b)

Mean True Error and Fraction of Trials with R>0.05

m	Mean True Error	Fraction of Trials with R>0.05
10.0	0.13745765011253883	0.992
20.0	0.1141803665565723	0.958
50.0	0.06489335676482527	0.6615
100.0	0.03712845632976773	0.2145
200.0	0.019107486727950134	0.003
500.0	0.008030727577268948	0.0
1000.0	0.003952591356666191	0.0

c)




```
✓ ### c - finding which degree SRM selects - Joe ···

Degree selected by SRM: 1
```

```
✓ ### c - finding which degree 5-fold cross validation selects - Joe ···

Degree selected by 5-fold CV: 15
```

```
✓ ### c - find which minimizes test error- Joe ···

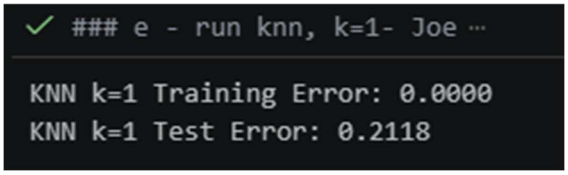
Degree selected by minimum test error: 9
```

SRM penalizes model complexity, so it selects the simplest model. Since cross-validation evaluates using training data, it will prefer a more complex model, selecting the highest degree 15. Using the test error, it selects a degree around the midpoint, 9.

d)

The numerical value of the bound is around 0.7-0.8, which is not informative since it remains over 0.5. When used quantitatively, it is not as informative as the train/test error, however qualitatively it is still useful to show the tradeoff between model complexity and training error.

e)



On the training data, the 1-NN can memorize each labeling, and performs perfectly on it. Also, 1-NN performs way better than random guessing on the test data, with some of the error being irreducible. Since (1) does not say anything about an infinite dimension, and cannot provide a guarantee that is meaningful, it means neither that the classifier will perform well or badly on generalizing to new data.

Part 2 – Essay – Prompt B - Max

VC theory gives us some insight into how machine learning models can generalize. The VC Dimension shows us how complex, powerful and capable our hypothesis space is. It is the maximum number of points that can be shattered. If the hypothesis space has a finite VC Dimension, we know our model will have a true risk close to the empirical risk with high probability for sufficient samples. A hypothesis space having an infinite VC Dimension does not always mean that it will fail to generalize.

The 1-NN method of classification works by choosing the class of the nearest training point to the new point. This can perfectly mimic the labeling of any distinct points in the training data, so it will have an infinite VC Dimension as it can shatter any finite set. As the number of samples increases, the error of the 1-NN converges to at most twice the Bayes error (Cover & Hart, 1967). This is because as the number of data points increases, the nearest neighbor to each point will be very close and, with a reasonably smooth distribution, will likely share the same class.

One may think that the 1-NN method does not fall within VC theory due to being a non-parametric method. However, the family of sign(sin(ax)) only has one real parameter but has an infinite VC Dimension (Shalev-Shwartz & Ben-David, 2014). We can see that the number of parameters in a hypothesis is not a good true measure of its capacity. The VC Dimension only measures the worst-case expressiveness of the class, and the "bad"

hypotheses require a very large frequency. If we choose a learner that prefers small values of a, we will get good generalization for slowly varying data. We do not care about all the hypotheses in the class, only those that the algorithm will reach.

Modern neural networks have many more parameters than training examples. The VC Dimension will increase with the number of parameters and therefore the bounds will be greater than 1, providing no useful information. Common architectures trained using SGD (Stochastic Gradient Descent) can perfectly fit the CIFAR-10 images with randomly assigned labels (Zhang et al., 2017), meaning that the class can memorize anything. Yet when trained on the true labels, the same networks generalize well. Since the hypothesis class is the same in both cases, any bound that only depends on H should give the same guarantees, meaning that the bounds cannot explain why one will generalize and the other will not.

VC bounds are a worst-case scenario. They consider the worst hypothesis in H and the worst distribution, rather than the hypothesis the algorithm picks or the data it sees. There are various frameworks that, instead of looking at the size of the hypothesis space, look at a data or algorithm dependent measure. Margin bounds look at how complex the hypothesis is based on the margin achieved on the training data with respect to the scale of the weights. Bartlett, Foster and Telgarsky (2017) derived bounds based on the spectral norm of the weight matrices scaled by the margin and showed that these quantities differ between true and random labels. PAC-Bayes bounds look at how complex the hypothesis is based on the KL divergence between the posterior distribution over the hypothesis space and the prior (which is fixed before seeing the data). Large models can have a low divergence if the solution is robust to weight changes. This was used by Dziugaite and Roy (2017) to produce non-vacuous bounds for real neural networks.

Stability looks at how much an algorithm will change its output given a change in one of the training examples. Bousquet and Elisseeff (2002) showed that stable algorithms will generalize. Hardt, Recht and Singer (2016) showed that using a limited number of steps of SGD will result in a stable algorithm. Stability is an algorithm dependent measure and not related to H. Other algorithm-dependent analyses, such as studies of gradient descent's implicit bias toward low-norm or max-margin solutions, follow the example of sign(sin(ax)). We only need to look at the hypotheses that the algorithm can reach.

VC theory can help us understand why models generalize when they have a small capacity, but it provides no information when they have a large capacity. 1-NN, sign(sin(ax)) and over-parameterized networks show that models with an infinite or very large VC Dimension can still perform well. Margin bounds, PAC-Bayes, stability and algorithm dependent analyses all provide data or algorithm dependent bounds instead of

distribution-free bounds that only depend on the class. This helps greatly reduce the gap; however, there is no single framework that provides a complete explanation as to why deep networks can generalize.

References

- Bartlett, P. L., Foster, D. J., & Telgarsky, M. (2017). Spectrally-normalized margin bounds for neural networks. *Advances in Neural Information Processing Systems 30*.
- Bousquet, O., & Elisseeff, A. (2002). Stability and generalization. *Journal of Machine Learning Research, 2*, 499–526.
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27.
- Dziugaite, G. K., & Roy, D. M. (2017). Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *Proceedings of UAI 2017*.
- Hardt, M., Recht, B., & Singer, Y. (2016). Train faster, generalize better: Stability of stochastic gradient descent. *Proceedings of ICML 2016*.
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.
- Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2017). Understanding deep learning requires rethinking generalization. *Proceedings of ICLR 2017*.

Bonus – Necessary vs Sufficient Sample Size

Thm 2, is h > 1+ 32ln(1/delta), substituting delta=0.02, we get 126.185. Therefore the VC integer dimension satisfying this is 127.

For the sample size, the lower bound is m \approx (h-1) / 32\eps , substituting h=127, eps=0.01. This gives us 393.75, or 394 to satisfy.

$$+\underbrace{\sqrt{\frac{h\left(\ln\frac{2m}{h}+1\right)-\ln\frac{\eta}{4}}{m}}}_{=:f_m(h)\ \text{(VC confidence)}},$$

The VC confidence bound is where fm(127) with n=0.02. Solving this using scipy.optimize.brentq, shown in the code at the bottom, gives us

```
Necessary M: 394
Sufficient M: 17210684.487011913
Pessimistic Factor: 43681.9403
```

The gap is extremely large which means the VC confidence bound is extremely conservative to guarantee the specified error.